

Claude Code in 90 Days — a free self-study course

Learn to get the most out of Claude Code — from your first command to your own agents and automation. 80% practice, 20% theory. No certificate, but something better: 6 finished mini-projects.

Who it's for: anyone who knows terminal basics (open a terminal, `cd` into a folder) and wants to learn to build software with an AI agent. You don't have to be a professional developer — Claude writes the code, you learn to steer it.

Prerequisites:

- Claude Code installed (CLI or desktop app) and a Claude account
- Git installed (needed from Module 6, but install it right away)
- Any operating system (Windows / Mac / Linux)

How to use this course:

- 12 weeks, 6 modules (each module = 2 weeks)
- Rhythm: 5 days × 45–60 minutes of practice + 1 retrospective day (what did I learn, what is still unclear)
- Each module builds a **mini-project from scratch** — you don't need any existing code
- Exercises are labeled **E** (e.g. E1.3 = Module 1, exercise 3). Every exercise has a solution at the end of the course in the **Solutions** section — but **try it yourself first**. Mistakes are part of learning; look at the solution only when you're stuck.
- Each module ends with a **Knowledge check** — a task you do without instructions, only with a goal. That's your "exam" instead of a certificate.

Module overview:

Module	Topic	Weeks	Mini-project
1	Claude CLI basics	1–2	Expense tracker (Python console app)
2	CLAUDE.md and memory	3–4	Personal portfolio site (HTML/CSS/JS)
3	Plan mode and Loop	5–6	Quiz app (web)
4	MCP servers	7–8	Weather + notes (external data + database)
5	Parallel subagents	9–10	Analyzing an open-source project
6	Git integration and automation	11–12	Final project of your choice

Module 1 — Claude CLI basics (weeks 1–2)

Mini-project: "Expense tracker" — a Python console application that records and sums your personal expenses. Starts from a completely empty folder.

WHY this exists

Claude Code is not a chat. A chat gives you text that you then copy into files by hand. Claude Code is an **agent**: it reads your files itself, edits them itself, runs commands in your terminal itself, sees the error it caused and fixes it itself. Your job shifts from "typing code" to "describing what you want and checking the result". The whole course is really about learning that new skill: **how to tell an agent precisely what you want and how to verify it actually did it.**

Module theory (read once, come back as needed)

Two working modes:

- **Interactive:** type `claude` in your project folder → a conversation opens that lasts until you close it. This is where you work 95% of the time.
- **One-shot:** `claude -p "question or task"` → Claude does it, prints the result and exits. For scripting and quick checks (more in Module 6).

Basic commands inside a conversation:

Command	What it does	Why you need it
<code>/help</code>	list of all commands	the first thing you type in a new tool
<code>/clear</code>	wipes the whole conversation	new task = clean context = better results
<code>/compact</code>	compresses the conversation to free space	long sessions fill the "context" — Claude has limited working memory
<code>/model</code>	switches the model (faster/cheaper vs. smarter)	you don't need the strongest model for small stuff
<code>/resume</code>	returns to an earlier session	continue tomorrow where you stopped
<code>/rewind</code>	rolls the conversation (and optionally files) back to an earlier point	"undo" when Claude goes in the wrong direction
<code>Esc</code>	interrupts Claude mid-work	see it doing the wrong thing? Don't wait for the end

Referencing files: type `@` followed by a file name (e.g. `@expenses.py`) — Claude is guaranteed to read that file. Without it, it decides on its own what to read (it usually guesses right, but `@` is a guarantee).

Permission modes — Claude asks for permission before changing anything or running anything. WHY: an agent that edits files and runs commands on its own must have a brake. Modes (switch with **Shift+Tab**):

- **default** — asks for every edit/command. Start here: you see exactly what it does.
- **acceptEdits** — file edits pass automatically, commands still ask. Once you've built trust.
- **plan** — Claude may only READ and make a plan, changes nothing (Module 3).

- **bypassPermissions** — asks nothing. Only for isolated/safe environments — skip it for now.

Week 1 — first steps and a first application

Day 1 — installation and first conversation

1. Create an empty folder `expense-tracker` and enter it in the terminal.
2. Run `claude`. Type `/help` and read what exists (you don't have to memorize anything).
3. **E1.1:** Ask Claude to create a CSV file with 10 made-up expenses (date, description, category, amount) and a Python script `expenses.py` that reads that CSV and prints the total. Run the script. **Solution**
4. Pay attention: Claude asked for permission before writing the file and before running the command. Read WHAT exactly it is asking before you approve — that habit is worth gold.

Day 2 — extension: entering new expenses

1. Continue in the same folder (`claude` , and if you want the old conversation: `/resume`).
2. **E1.2:** Ask for a menu: an option to enter a new expense (appended to the CSV) and an option to show the total. Test both. **Solution**
3. Try `@expenses.py` in a message — see how Claude reacts when you hand it a file explicitly.

Day 3 — monthly report

1. **E1.3:** Ask for a report by month and by category (e.g. "food: 45.00, transport: 12.00"). Let Claude propose the output layout itself, then refine it: "align the columns", "add a total at the bottom". **Solution**
2. The point of the exercise: **you don't have to know the exact layout up front — refine it through conversation.** That's a normal way to work.

Day 4 — deliberately causing an error (debugging)

1. **E1.4:** Open the CSV in an editor by hand and break one row (e.g. delete the amount or type a letter instead of a number). Run the script — it will crash. Then type into Claude only: "run expenses.py and fix the error". Watch: it will read the error, find the cause, fix it and run again to verify. **Solution**
2. WHY this is key: this is the loop you'll use constantly — **Claude can check its own work if you give it a way to check it** (running, a test, an example).

Day 5 — one-shot mode

1. Exit the conversation. **E1.5:** From the terminal run: `claude -p "how many rows does expenses.csv have and what is the single largest expense?"` **Solution**
2. Try 2–3 more one-shot questions about your code. Notice the difference: no conversation, you get an answer and it's over.

Day 6 — retrospective: Write down (in a notebook or a `NOTES.md` file): what surprised me? When did Claude get it wrong and why — was my description imprecise? Which command haven't I tried yet?

Week 2 — sessions, context and permission modes

Day 1 — resuming sessions

1. Close the terminal completely. Open it again, enter the folder, run `claude --resume` (or `claude` then `/resume`) and pick yesterday's session.

2. **E1.6:** Continue EXACTLY where you stopped: ask for a category "other" for expenses entered without a category. Notice that Claude "remembers" everything from that session. [Solution](#)

Day 2 — context and /compact

1. Theory in 3 sentences: Claude has a limited "working memory" (context). Everything you've said and every file it has read fills it. When it fills up, older parts get summarized — that's why long sessions become "forgetful".
2. Practice: look at the context indicator (bottom of the screen). Type `/compact` and see what happens. Rule to take away: **new task** → `/clear`. Don't drag one conversation on for days.

Day 3 — /rewind and correcting course

1. **E1.7:** Ask for something deliberately vague, e.g. "tidy up the output a bit". When Claude does something you don't like, use `/rewind` to go back, then repeat the request precisely ("use a table with date/description/amount columns, amount right-aligned"). Compare the results. [Solution](#)
2. Lesson: the quality of the result = the quality of the description. `/rewind` + a better description is often faster than 5 rounds of fixes.

Day 4 — permission modes in practice

1. **E1.8:** Switch to **acceptEdits** (Shift+Tab) and ask for a small improvement (e.g. a nicer date in the output). Notice that file edits no longer ask for confirmation, but commands still do. Switch back to default. [Solution](#)
2. Think and write down: in which situations would acceptEdits be dangerous? (e.g. a folder with important files and no git)

Day 5 — KNOWLEDGE CHECK for Module 1 E1.9 (no instructions): In a new session (`/clear`), using everything you've learned, add exporting the monthly report to a JSON file. Conditions: (1) you must use an `@` reference at least once, (2) you must let Claude run and verify the code itself, (3) at the end, check the JSON file with a one-shot `claude -p` command. [Solution and success criteria](#)

Day 6 — retrospective.

Module 2 — CLAUDE.md and memory (weeks 3–4)

Mini-project: "Personal portfolio site" — a static site (HTML/CSS/JS) with a home page, a projects page and a contact form. From an empty folder.

WHY this exists

Claude starts every new session **blank** — it knows nothing about your project, your rules, your style.

`CLAUDE.md` is a file Claude automatically reads at the start of EVERY session in that folder. It's the difference between a coworker you re-explain everything to every single day — and a coworker who read the "house rules" before sitting down.

Module theory

Three levels of CLAUDE.md:

File	Applies to	Example content
<code>~/ .claude/CLAUDE.md</code> (global)	ALL your projects	"answer me in English", "keep commit messages short"
<code>CLAUDE.md</code> in the project folder	that project, and everyone working on it (goes into git)	project rules, conventions, known pitfalls
<code>CLAUDE.local.md</code>	that project, only your machine (doesn't go into git)	your personal shortcuts, local paths

What BELONGS in CLAUDE.md: rules that can't be seen from the code. Conventions ("all site copy is in English"), prohibitions ("no frameworks, plain HTML/CSS/JS"), pitfalls ("don't touch file X, it's auto-generated"), project commands ("the site is tested by opening index.html").

What does NOT belong: things Claude can see from the code itself (the file list, what each function does) — that goes stale and fills the context for nothing.

The # shortcut: type `#` at the start of a message (e.g. `# always use css variables for colors`) — Claude writes it permanently into memory/CLAUDE.md for you.

`/init`: a command that analyzes an existing project and drafts an initial CLAUDE.md on its own.

Week 3 — the pain without CLAUDE.md, then the cure

Day 1 — a site without rules (on purpose)

1. New folder `portfolio`. **E2.1:** Ask for a home page: name, short bio, 3 sections (About / Projects / Contact). Do NOT give any rules about style/language. Just accept what it offers. [Solution](#)

Day 2 — "forgetfulness" in action

1. Close the session. Open a NEW one (without `--resume`!). **E2.2:** Ask for a new page `projects.html` with 3 project cards. Compare it with the home page: colors? font? language of the copy? structure? Odds are something clashes — Claude had no way of knowing your conventions. Write down what exactly clashes. [Solution](#)

Day 3 — writing CLAUDE.md

1. **E2.3:** Create a `CLAUDE.md` in the project folder (you can also ask Claude to write it from your dictation) with at least 5 rules, e.g.: the language of all copy, a color palette (concrete hex values), no frameworks or libraries, every page uses a shared `style.css`, lowercase file names. [Solution with a complete example file](#)

Day 4 — testing the cure

1. New session (not resume). **E2.4:** Ask for a third page (`contact.html` with a form). Do NOT mention a single rule. Check: did it follow `CLAUDE.md` without being reminded? If some rule wasn't followed — the rule is probably written unclearly; sharpen the wording. [Solution](#)

Day 5 — the # shortcut and /init

1. **E2.5:** In the middle of working, type a `#` rule you were missing (e.g. `# buttons always have a hover effect`). Look where it wrote it. Then, in some OTHER existing folder (any old project or a downloaded repo), run `/init` and read what it generated — what would you remove/add? [Solution](#)

Day 6 — retrospective.

Week 4 — hierarchy and maintaining memory

Day 1 — the global CLAUDE.md

1. **E2.6:** Create/extend the global `~/ .claude/CLAUDE.md` (on Windows: `C:\Users\<you>\.claude\CLAUDE.md`) with 2–3 PERSONAL rules you want everywhere (communication language, commit message style...). Open a session in the portfolio project and verify BOTH files apply — global and project. [Solution](#)

Day 2 — what doesn't belong in CLAUDE.md

1. A thinking exercise: add a deliberately bad entry to `CLAUDE.md` — a detailed description of what every file does. Over the next sessions notice that it (a) goes stale as soon as the code changes, (b) wastes context. Delete it. Rule: **CLAUDE.md contains what can NOT be seen from the code.**

Day 3–4 — improving the site with a living CLAUDE.md

1. Add whatever you like to the site (gallery, dark theme, animations). Every time you have to REMIND Claude of something — that's a candidate for `CLAUDE.md`. Keep extending it as you go (the `#` shortcut).

Day 5 — KNOWLEDGE CHECK for Module 2 E2.7 (no instructions): Simulate a "new team member": wipe all sessions from your head — open a completely new session and ask: "add a page with a price list of services". Success criterion: the new page fits the site (language, colors, structure, style) WITHOUT a single reminder from you. If not — `CLAUDE.md` has a hole; find it and patch it. [Solution and criteria](#)

Day 6 — retrospective.

Module 3 — Plan mode and Loop (weeks 5–6)

Mini-project: "Quiz app" — a web quiz: questions with 4 answers, timing, a final score and a high-score table (localStorage). From an empty folder.

WHY this exists

For small changes Claude can start right away — if it errs, the fix is cheap. For BIGGER undertakings (a new app, refactoring, a change touching 10 files) a wrong direction means an hour of cleanup. **Plan mode** flips the order: Claude may first only READ and investigate, then proposes a plan to you, and only when you approve the plan — may it make changes. Reviewing a 20-line plan is immeasurably cheaper than reviewing 500 lines of wrong code.

Module theory

- You enter plan mode with **Shift+Tab** (until you see "plan mode"), or by starting `claude` and switching before the first request.
- In a plan, Claude explores the code, asks questions and finally proposes a plan. You can: **approve**, **request changes** (just write what to change — the plan gets revised), or **abandon it**.
- When plan mode is NOT needed: small stuff ("change the button color"). Overblown ceremony for small things slows you down.
- **Extended thinking:** for hard problems add "think hard" to the request — Claude gets more "thinking time" before answering. Use for: architecture design, weird bugs, decisions with multiple options.
- `/loop` : repeats a task at intervals (e.g. check something every 5 minutes). For monitoring and recurring chores.

Week 5 — the quiz through plan mode

Day 1 — a plan for the structure

1. New folder `quiz`. Enter plan mode. **E3.1:** Ask: "build a web quiz app: questions with 4 answers, 10 general-knowledge questions, score at the end". READ the plan carefully before approving: does it mention the files you expect? Is anything overcomplicated? [Solution](#)
2. Approve the plan and let it implement. Open the quiz in a browser.

Day 2 — deliberately rejecting a plan

1. Plan mode. **E3.2:** Ask for a high-score table (player name + score + date, saved in localStorage, top 10 shown). When the plan arrives — do NOT approve immediately: request at least 2 changes (e.g. "scores should also be tracked per category" or "add a button to clear the scores"). Watch the plan get revised and only then approve. [Solution](#)
2. Lesson: the plan is a PROPOSAL, not an obligation. The biggest value of plan mode is negotiating BEFORE the code.

Day 3 — timing + think hard

1. **E3.3:** Ask for a per-question timer (e.g. 15 seconds, time out = wrong answer) and bonus points for speed. Add to the request "think hard about the edge cases: what if the user switches tabs, what if they answer in the last second". Compare the plan's quality with a plain one. [Solution](#)

Day 4 — when plan mode is NOT needed

1. Without plan mode, directly: 3–4 small changes ("bigger question font", "green/red answer coloring", "add 5 more questions"). Feel the difference in tempo. Write down your rule of thumb: from what task size is plan mode worth it FOR YOU?

Day 5 — /loop

1. **E3.4:** Try `/loop` on a harmless task, e.g.: "every 5 minutes check whether the folder has new .html files without a corresponding link in index.html and tell me". Let it run 2–3 rounds, then stop it. Think: what in YOUR real work would be worth monitoring like this? [Solution](#)

Day 6 — retrospective.

Week 6 — a big refactor through a plan

Day 1–2 — preparation and the refactoring plan

1. Your quiz is probably in 1–2 files. **E3.5:** In plan mode ask for a refactor into modules: separate files for questions (data), quiz logic, timer, scores and rendering. A condition in the request: "after the refactor everything must work identically to before". Review the plan IN DETAIL — refactoring is the easiest place to break something along the way. [Solution](#)
2. Approve and let it run. Test ALL quiz features afterwards (play a full game, check the scores).

Day 3–4 — improvements on the new structure

1. Add question categories and a category picker at the start. Notice: is the change EASIER on the modular structure? That was the point of the refactor.

Day 5 — KNOWLEDGE CHECK for Module 3 E3.6 (no instructions): Through plan mode, from plan to implementation: add a "2 players take turns" mode with a side-by-side result at the end. Criteria: you revised the plan at least once before approving; after implementation you personally tested both modes (1 and 2 players). [Solution and criteria](#)

Day 6 — retrospective.

Module 4 — MCP servers (weeks 7–8)

Mini-project: "Weather + notes" — an app that shows the weather forecast (external data) and your daily notes (a local SQLite database).

WHY this exists

Out of the box Claude "sees" only two things: your files and your terminal. It doesn't see your database, your browser, your Slack, your GitHub account. **MCP (Model Context Protocol)** is a standard way to give it new "senses" — an MCP server is a small program that exposes tools to Claude ("read the table", "open a page in the browser", "send a message"). Instead of you retelling Claude what's in the database, it looks for itself.

Module theory

Commands:

- `claude mcp add <name> ...` — adds a server (the exact syntax depends on the server; every server's docs have a ready-made command to copy)
- `claude mcp list` — list of connected servers and their status
- `claude mcp remove <name>` — removal
- `/mcp` inside a session — server status and sign-in (for servers that need an account)

Scope — where the configuration is remembered:

Scope	Applies to	When
<code>local</code> (default)	only you, only that project	trying things out, personal tools
<code>project</code>	everyone on the project (the <code>.mcp.json</code> file goes into git)	the team shares the same tools
<code>user</code>	you, in all projects	tools you use everywhere

Security note (important!): an MCP server has access to whatever you give it. Add only servers you trust (official, well-known). A server that reads your database — think about whether it also needs write access.

Week 7 — the first MCP: a database

Day 1 — the app (without MCP, to start)

1. New folder `weather-notes`. **E4.1:** Ask for an app (Python plus a bit of web is fine, let Claude propose): a SQLite database `notes.db` with a notes table (date, text), entering and viewing notes per day. Insert 5–6 sample notes. **Solution**

Day 2 — a SQLite MCP server

1. **E4.2:** Find and add an MCP server for SQLite (ask Claude itself: "what's the recommended way to add an MCP server for sqlite, and give me the exact command for my notes.db database"). After adding: `claude mcp list` must show it. **Solution**
2. Test question for Claude: "how many notes are in the database and which is the newest?" — it must answer by reading THROUGH MCP, not by writing a script.

Day 3 — feeling the difference

1. **E4.3:** Do the same task both ways and compare: (a) ask Claude something about the data WHILE the MCP is active; (b) remove the server (`claude mcp remove`), then ask the same — now it will have to write a script/command to reach the data. Write down the difference in speed and number of steps. Put the server back. [Solution](#)

Day 4 — external data (the forecast)

1. **E4.4:** Add a weather forecast for your city to the app (there are free APIs without registration — let Claude propose and implement the call FROM the app's CODE). [Solution](#)
2. A discussion with yourself (write it down): the forecast is fetched by the app's CODE via an API, and the database is read by CLAUDE via MCP. What's the difference? (Answer in solution E4.7.)

Day 5 — retrospective + tidying up.

Week 8 — the browser MCP and the registry

Day 1–2 — the browser MCP

1. **E4.5:** Connect a browser MCP (e.g. the "Claude in Chrome" extension or a Playwright MCP — ask Claude what's available for your environment and how to install it). Task: have Claude ITSELF open your app in the browser, enter a new note through the form and verify it appeared in the list. [Solution](#)
2. WHY this is big: until now YOU were Claude's eyes for everything happening in the browser. Now it can click and read the page itself — it can test what it builds on its own.

Day 3 — the MCP server registry

1. **E4.6:** Explore what exists: ask Claude about well-known/official MCP servers, or search "MCP servers registry" in a browser. Pick ONE that seems personally useful to you (GitHub? file system? something else?), add it and invent a mini-task to try it. [Solution](#)

Day 4 — .mcp.json for sharing

1. Add one of your servers with `--scope project` and look at the resulting `.mcp.json` file in the project. That's the file a colleague would get through git and have the same tools as you.

Day 5 — KNOWLEDGE CHECK for Module 4 E4.7 (no instructions, theory + practice): (1) Explain in your own words, in 5 sentences, the difference between an MCP server and an API call from the app's code — and give one example each of when which is the right choice. (2) Practical: have Claude, using the browser MCP + the SQLite MCP together, test the whole app and write you a short report of what works and what doesn't. [Solution and criteria](#)

Day 6 — retrospective.

Module 5 — Parallel subagents (weeks 9–10)

Mini-project: analyzing someone else's open-source project + a custom "reviewer" agent that reviews your quiz from Module 3.

WHY this exists

Two problems subagents solve:

1. **Context is limited.** If Claude has to read 50 files to answer one question, those 50 files fill your session forever. A subagent is a "scout": it goes off, reads everything in ITS own separate context, and brings you back only a 10-line conclusion.
2. **Some jobs are independent.** Three questions about three different parts of the code don't have to wait for each other — three subagents work in parallel.

When they're NOT worth it: small tasks. A subagent starts "from zero" (it doesn't see your conversation), so for a small thing it spends more on getting oriented than on the work.

Module theory

- You launch subagents with natural language: "send a subagent to investigate X", "run 3 agents in parallel: the first one should..., the second..., the third...".
- Built-in types: **Explore** (read-only — a safe scout), **Plan** (designs an approach), **general-purpose** (can do everything).
- **Custom agents** — your own specialists: a file in `.claude/agents/` with a role description, instructions and (optionally) restricted tools. They can also be created with the `/agents` command. WHY: instead of typing the same long prompt every time ("review the code and check these 10 things"), you write it down once as an agent.

Week 9 — scouts on someone else's code

Day 1 — cloning the target

1. Pick a well-known open-source project of medium size and clone it, e.g.: `git clone https://github.com/pallets/flask` (or any project that interests you). Enter the folder, run `claude`.

Day 2 — one scout

1. **E5.1:** Send one Explore agent: "send a subagent to investigate how error handling is organized in this project and bring me back a short report with the paths of the key files". Notice: your session stayed "light" — the agent read in its own context. **Solution**

Day 3 — three parallel scouts

1. **E5.2:** One request, three agents in parallel, three different questions about the project (e.g.: how tests are written / how configuration is organized / which external libraries it uses and what for). Ask Claude to merge the findings into one overview at the end. **Solution**

Day 4 — comparison with/without

1. **E5.3:** Ask the same kind of question (a new question about the project) once DIRECTLY (no subagent, let it read files in your session) and once THROUGH a subagent. Compare: context usage (the

indicator), time, answer quality. Write down when you'd pick which approach. [Solution](#)

Day 5 — retrospective.

Week 10 — your own specialist

Day 1–2 — the custom "reviewer" agent

1. **E5.4:** Create a custom agent in `.claude/agents/reviewer.md` (you can use `/agents` or ask Claude to create it). Role: reviews code against a checklist — readability, variable names, code duplication, obvious bugs, comments where truly needed. Make it READ-ONLY (only reads, changes nothing) and have it return the report as a list of findings with a path and a suggestion. [Solution with a complete example file](#)

Day 3 — the reviewer at work

1. **E5.5:** Go back to the quiz folder from Module 3 (copy the agent over or create it at user level). Run the reviewer on the whole quiz. Read the findings CRITICALLY: what do you agree with, what not? Fix 2–3 findings (you can also ask regular Claude for the fixes). [Solution](#)

Day 4 — tuning the agent

1. The reviewer surely reported something irrelevant too. Refine its instructions (e.g. "don't report style nitpicks unless they're agreed in CLAUDE.md") and run it again. WHY: an agent gets calibrated through iterations, like a coworker.

Day 5 — KNOWLEDGE CHECK for Module 5 E5.6 (no instructions): On an open-source project of your choice: (1) three parallel Explore agents with three questions YOU come up with, (2) a merged report, (3) your own assessment in 5 sentences: what would be hardest to change in this project and why — based on the report. Criterion: the report contains concrete file paths, and your assessment cites the findings.

[Solution and criteria](#)

Day 6 — retrospective.

Module 6 — Git integration and automation (weeks 11–12)

Final project: a complete small application of YOUR choice — from idea to a GitHub repository, applying everything you've learned.

WHY this exists

So far you've been producing code. This module is about how code LIVES in real work: versions (git), reviewing changes before merging (a review — finding a bug in review is cheaper than in production), and automation (hooks, your own commands, headless mode) — so the repetitive part of your work gets packaged and runs on its own.

Module theory

- **Git through Claude:** Claude can do the whole git flow — `git init`, branches, meaningful commit messages (it reviews the changes itself and composes the message), and with the **GitHub CLI** (`gh`) also PRs. You still APPROVE every command.
- `/code-review` — reviews your current changes and reports bugs/problems before you commit them.
- **Hooks** — automatic actions triggered on events (e.g. "after every file edit, run the tests"). Defined in `.claude/settings.json`. The important WHY: a hook is executed by the tool itself (guaranteed, every time), not by Claude at its own discretion — rules that MUST happen go into a hook, not into CLAUDE.md.
- **Custom slash commands** — your workflow as a command: a file in `.claude/commands/` (e.g. `ship.md`) becomes `/ship`. Instead of typing the same sequence of requests every time.
- **Headless mode** — `claude -p "task"` from a script/CI: Claude as a tool in automation, without a conversation.

Week 11 — a git flow from scratch

Day 1 — the repo and the first commit

1. Pick an idea for the final project (suggestions: a shared shopping list, a workout log, an RSS news reader, a task planner — or something of your own). New folder.
2. **E6.1:** Right at the start: ask Claude for `git init`, a `.gitignore` for the project type, and a CLAUDE.md (Module 2 knowledge!). Then the first feature and the first commit — but READ the commit message it proposes before approving. **Solution**

Day 2 — a branch + a feature + a review

1. **E6.2:** A new feature entirely on a branch: "create a branch for feature X, implement it" → then BEFORE the commit run `/code-review`. Read the findings, fix the justified ones, then commit. **Solution**

Day 3 — GitHub and a PR

1. Create a (free) GitHub account if you don't have one and install the `gh` CLI (`gh auth login`).
2. **E6.3:** Ask Claude to publish the repo on GitHub and open a Pull Request from your branch, with a description of the changes. Look at the PR in the browser — that's the format teams use to exchange code. Merge it. **Solution**

Day 4 — a hook

1. **E6.4:** Set up a hook that runs after every Claude edit of a file — to start with, something simple and visible (e.g. for a Python project: a syntax check of the edited file). Test it: request an edit and make sure the hook fired. [Solution with a settings.json example](#)
2. Think: which rule from your CLAUDE.md should actually be a hook? (Reminder: CLAUDE.md = a request, a hook = a guarantee.)

Day 5 — retrospective.

Week 12 — your own command, headless, and the FINAL EXAM

Day 1 — a custom slash command

1. **E6.5:** Create `.claude/commands/ship.md` — a `/ship` command that: runs the code checks, composes a commit message from the changes, commits and pushes. Try it on a real change. [Solution with a complete example file](#)

Day 2 — headless

1. **E6.6:** Write (with Claude's help) a script that uses `claude -p` so it can answer a question about the project daily, e.g.: "list the TODO comments in the code and propose the order to resolve them". Run it by hand and look at the output. [Solution](#)

Day 3–5 — FINAL EXAM **E6.7:** Finish your project applying EVERYTHING — checklist:

- ☐ CLAUDE.md exists from the start and is kept up to date (Module 2)
- ☐ At least one bigger feature done through plan mode (Module 3)
- ☐ At least one MCP server in use (Module 4)
- ☐ At least one investigation through a subagent (Module 5)
- ☐ The full git flow: branches, review before commit, a PR on GitHub (Module 6)
- ☐ At least one hook or custom command in the project (Module 6)
- ☐ The application WORKS — you tested it personally end to end

Success criteria

Day 6 — final retrospective: what from this course stays in your daily work? Write your own "top 5 rules for working with Claude Code".

After the course

- **Official documentation:** <https://code.claude.com/docs> — especially the parts on hooks, skills and the SDK
 - **Anthropic's best-practices articles** on agentic coding (search "Claude Code best practices")
 - Inside the tool itself: `/help` and questions like "how do I do X in Claude Code" — the tool can explain itself
 - Ideas to continue: your own skills (`.claude/skills/`), the Claude Agent SDK (building your own agents), more advanced MCP integrations
-
-

SOLUTIONS

Rule: look at a solution only after at least 15 minutes of your own attempt. The prompts below are REFERENCE prompts — your wording doesn't have to match, it just has to carry the same key information. If your result works but differs from the solution — that's a success, not a mistake.

Module 1 — solutions

Solution E1.1

Reference prompt:

Create a CSV file `expenses.csv` with columns `date`, `description`, `category`, `amount` and 10 made-up rows (dates from the last 2 months, categories: food, transport, bills, fun). Then create a Python script `expenses.py` that reads that CSV and prints the total of all amounts. Run it to verify it works.

Expected outcome: two files in the folder; Claude ran the script itself and showed you the printed total.

Common mistakes:

- *A prompt without a column structure* ("make me some CSV with expenses") — you get random columns you'll have to rework later. The earlier you pin down the data structure, the fewer reworks later.
- *Not asking it to run the script* — Claude may then hand over unverified code. Habit: "...and run it to verify" at the end of the request.

Solution E1.2

Reference prompt:

Add a menu to `expenses.py` with options: 1) enter a new expense (asks for date, description, category, amount and appends a row to the CSV), 2) show the total, 3) exit. After the change, run it and test entering one expense.

Expected outcome: the menu works; the new expense shows up in the CSV file. **Common mistakes:** asking for the three options in three separate messages — it works, but it's slower; ask for related things together. The opposite extreme — 10 features in one message — also hurts (harder to verify). Golden mean: one rounded-off unit per request.

Solution E1.3

Reference prompt (first step):

Add a 'monthly report' option: for a chosen month print the total per category and the grand total. Propose the output format yourself.

then refining through conversation: Right-align the amounts and add a line above the grand total.

Expected outcome: a readable report; you went through at least one round of refinement. **The point:** "propose it yourself" is a legitimate technique — Claude often proposes better than you would have specified, and you then just correct it.

Solution E1.4

Reference prompt: Run `expenses.py` – it crashes. Find the cause and fix it, then run it again to confirm. **Expected outcome:** Claude runs it, reads the traceback, finds the broken row (or writes more robust code that skips/reports bad rows), and PROVES the fix by running again. **Common mistakes:**

- Retelling the error in your own words instead of letting it see it — the exact information gets lost. Let it RUN and see the original.
- Accepting the fix without a re-run — always ask for proof.

Bonus question to think about: is it better for the script to crash on a bad row or to skip it silently? (Answer: it depends — but "silent" skipping without any message is the worst option; ask for at least a warning.)

Solution E1.5

Command: `claude -p "how many rows does expenses.csv have and what is the single largest expense?"` **Expected outcome:** an answer in the terminal, without entering a conversation. **When to use:** quick checks, scripts, automation. For anything with follow-up questions — interactive mode is better.

Solution E1.6

Procedure: `claude --resume` → pick yesterday's session → Add a category 'other': every expense entered without a category gets it automatically. **Expected outcome:** Claude continues with full knowledge of the context — you don't have to explain the project. **Common mistake:** starting a plain new session (`claude`) and being surprised it "doesn't know" what you're talking about. New session = blank memory (except `CLAUDE.md` — Module 2!).

Solution E1.7

Procedure: after a vague request and a bad result → `/rewind` → pick the point before the change → a precise request:

Rearrange the expense output into a table with columns: date (dd.mm.yyyy), description (max 30 chars), category, amount right-aligned with 2 decimals. Separate the header with a line.

The point: compare what you got from "tidy up the output a bit" versus the precise description. The difference you see = the value of precise describing.

Solution E1.8

Procedure: Shift+Tab until it says `acceptEdits` → request a small change → notice the file edit passes without a question, while running the script still asks. **Answer to when it's dangerous:** a folder without git (no way back), files that aren't just code (documents, configs with passwords), working on someone else's/production code. Rule: `acceptEdits` only once there's a safety net (git) and trust in the task.

Solution E1.9

Success criteria (check yourself):

- ☐ There's a new menu option that produces a JSON file of the monthly report
- ☐ The JSON contains: month, totals per category, grand total
- ☐ You used `@expenses.py` at least once in a request
- ☐ Claude ran and verified the code itself at least once
- ☐ You checked the result with `claude -p "read report-<month>.json and tell me whether it's valid JSON and whether the per-category totals add up to the grand total"`

Reference prompt for the main part:

```
@expenses.py Add an 'export to JSON' option: for a chosen month create a
file report-YYYY-MM.json with the structure {month, per_category: {...},
total}. Run it, export one month and show me the file contents.
```

Common mistake: doing everything in the old, long conversation — the check asked for a NEW session (`/clear`) precisely so you see how much depends on context.

Module 2 — solutions

Solution E2.1

Reference prompt:

Create the home page of a personal portfolio site: `index.html` with 'About', 'Projects', 'Contact' sections and made-up content. Plain HTML and CSS.

Expected outcome: any decent-looking site. There's no mistake to make here — the point of the exercise is in what follows tomorrow.

Solution E2.2

Expected outcome: a new page that probably DIFFERS from the home page — a different color tone, maybe different language of the copy, maybe inline styles instead of the shared CSS. You wrote down at least 2 concrete differences. **If it happened to fit perfectly:** look at how — Claude probably read `index.html` and imitated it. That's good, but it's NOT guaranteed; `CLAUDE.md` is the guarantee. (Still do E2.3.)

Solution E2.3

A complete example `CLAUDE.md` for the portfolio project:

```
# Portfolio site

## Rules
- All site copy is in English.
- Plain HTML/CSS/JS – no frameworks, libraries or CDN links.
- All pages use the shared style.css – no inline styles and no <style> blocks in the HTML.
- Color palette (these only): background #1a1a2e, text #eaeaea, accent #e94560, secondary #16213e. Defined as CSS variables in :root – new colors are not introduced without agreement.
- File names: lowercase, no spaces (contact.html, not Contact Page.html).
- Every page has the same header with navigation – when adding a new page, add its link to the navigation ON ALL pages.
- The site is tested by opening index.html in a browser (no build step).
```

Why exactly like this: every rule is (1) verifiable, (2) something that is NOT obvious from the code or is easy to violate, (3) short. A bad example of a rule: "the site should look nice" — unverifiable.

Solution E2.4

Expected outcome: the contact page respects the language, colors, shared CSS and navigation — without reminders. **If something wasn't respected:** the rule is probably fuzzy. Example: "use our colors" (fuzzy) vs. a palette with hex values (clear). Rewrite the rule and test again in a new session — that's the normal CLAUDE.md tuning cycle.

Solution E2.5

Procedure for #: you type e.g. `# buttons and links always have a hover effect` → Claude asks/shows where it's writing it → verify the rule ended up in CLAUDE.md. **For /init:** the generated file usually contains: a project description, the structure, commands. A critical eye: is any of it obvious from the code (a candidate for deletion)? Is a pitfall missing that only you know (a candidate for adding)? `/init` is a STARTING POINT, not a finished product.

Solution E2.6

Example global `~/ .claude/CLAUDE.md` :

```
# My global rules
- Communicate with me in English.
- Commit messages: short, starting with a verb ("Add...", "Fix...").
- When you propose a solution, first one sentence WHAT you propose,
  then the details.
```

Check: in a session in the portfolio folder ask "which rules currently apply to you?" — it should list both the global and the project ones. A conflict between rules? The more specific one (project) generally wins — but best is not to create conflicts.

Solution E2.7

Success criteria:

- ☐ The price list page is in the language from your rules
- ☐ It uses only palette colors and the shared style.css
- ☐ Navigation updated on all pages
- ☐ You did NOT remind it of a single rule

If it failed: find the EXACT rule that was violated → read it with the eyes of someone who doesn't know the project → rewrite it more concretely → repeat the test in a new session. This cycle (test in a clean session → hole → patch) is also the professional practice for CLAUDE.md.

Module 3 — solutions

Solution E3.1

Reference prompt (in plan mode):

Build a web quiz app: 10 general-knowledge questions, each with 4 answers of which 1 is correct, one question shown per screen, a result at the end (X/10). Plain HTML/CSS/JS, works by opening index.html.

What to look for in the plan before approving: the list of files (you expect index.html, style.css, something for logic and question data), the order of steps, and that there's NOTHING unnecessary (build tools, a framework — you didn't ask for them). If the plan includes something you don't understand — ASK before approving; that's the point of plan mode.

Solution E3.2

Example of revising the plan: the plan arrives → you reply e.g.:

Change the plan: 1) scores should also store the date played,
2) add a 'clear all scores' button with a confirmation before deleting.

→ a revised plan arrives → only then approve. **Expected outcome:** you see the plan change through negotiation, without a single line of code written. **Common mistake:** approving the plan "pro forma" and requesting changes AFTER implementation — you've paid for implementing the wrong version.

Solution E3.3

Reference prompt:

Think hard, then make a plan: a 15-second timer per question (time out = wrong), bonus points for answer speed. Pay attention to the edge cases: switching tabs mid-question, answering in the last second, what happens to the timer between questions.

Expected outcome: a plan that EXPLICITLY addresses the listed edges (e.g. pausing the timer when unfocused, or a clear decision not to pause). Compare: without "think hard" and without listing the edges, the plan is typically shallower exactly at the edges. **The point:** listing the edge cases in the request is YOUR job — you know what worries you; "think hard" gives the space to think it through.

Solution E3.4

Reference procedure: `/loop` then a description of the task and interval (exact syntax: see `/help`). A harmless example: checking that every .html file has a link in the navigation. **Expected outcome:** the task

repeated at least twice; you stopped it without trouble. **Ideas for real life:** watching whether a build passed; a periodic check of TODO comments; checking that all tests are green after a series of changes.

Solution E3.5

Reference prompt (plan mode):

```
Refactor the quiz into modules: questions.js (data only), quiz.js (flow logic), timer.js, scores.js (localStorage), render.js (DOM). index.html loads them. The app's behavior must remain IDENTICAL — nothing changes for the user. Make a plan that also includes how we'll verify nothing broke.
```

What to look for in the plan: steps that go piece by piece (not "everything at once"), and a VERIFICATION step at the end. **Success criterion:** after the refactor you played a full game + scores work + the timer works. A refactor without verifying behavior is not finished.

Solution E3.6

Success criteria:

- ☐ The whole flow through plan mode: request → plan → AT LEAST ONE revision from you → approval → implementation
- ☐ 2-player mode: alternating questions or alternating rounds (your choice — but made consciously at the plan stage)
- ☐ A side-by-side result at the end + the winner
- ☐ Personally tested: a game in 1-player mode AND a game in 2-player mode

Typical trap: 2-player mode opens questions the plan must answer (same questions for both? what about the high scores?) — if the plan didn't mention them, you should have requested that in the revision.

Module 4 — solutions

Solution E4.1

Reference prompt:

Build a 'weather and notes' app: a SQLite database notes.db with a table notes (id, date, text), and a simple interface (propose one – Python with a small web server is fine) for entering a note for a day and viewing notes per day. Insert 6 sample notes across the last week. Run it and show me how to open it.

Expected outcome: entering and viewing works; the database exists as a file.

Solution E4.2

Procedure: ask Claude for the exact command for your environment — MCP servers get installed by various commands and change over time, so "ask the tool" is more reliable than copying from a course. You'll typically get something of the form `claude mcp add <name> -- <command that starts the server with the path to the database>`. **Check:** `claude mcp list` shows the server as active; the question "how many notes are in the database?" gets answered through an MCP tool (you'll see the tool call in the session), not by writing a script. **Common mistake:** adding the server and asking in the OLD session — as a rule you need a new/restarted session for the server to be visible.

Solution E4.3

Expected observation: with MCP — one tool call, a direct answer. Without MCP — Claude writes and runs a script (more steps, more approvals, slower), but it ALSO reaches the answer. **The takeaway:** MCP is not the only way — it's the FASTER and CLEANER channel for things you do often. For a one-off query, a script is perfectly fine; for daily work with a database, MCP pays off.

Solution E4.4

Reference prompt:

Add a current weather display for [your city] to the app: use some free API without registration (propose one and justify the choice), show the temperature and conditions on the notes page. Also handle the case when the API doesn't respond – the app must not crash.

Expected outcome: the forecast on the page; the app survives even without internet (try it!).

Solution E4.5

Procedure: install a browser MCP per the instructions for your environment (ask Claude what's available). Then:

Open the app in the browser, enter a new note "test from the browser" for today through the form, and verify it appeared in the list. Report what exactly you did and what you saw.

Expected outcome: Claude clicks/types/reads the page itself and returns a report. **Criterion:** the note really exists in the database (check through the SQLite MCP — a nice opportunity to combine both).

Solution E4.6

There is no single correct solution — the criterion is the process: (1) you looked at what exists, (2) picked a server with a JUSTIFICATION of what it gives you, (3) added it and (4) performed a meaningful mini-task. Example: the GitHub MCP + "list the open issues on repo X and suggest which is the easiest for a beginner". **The security check you should have done:** is the server official/well-known? What can it do (read-only, or write too)?

Solution E4.7

Reference answer to the theory part (compare yours): An API call from the code is part of the APPLICATION — it executes every time the app runs, for every user (the forecast MUST be an API call from the code, because the app's users don't have Claude). An MCP server is a tool for CLAUDE during DEVELOPMENT — it gives it eyes/hands into external systems while it helps you (reading the database during development is ideal for MCP, because users don't need that, and it speeds up you and Claude). Rule of thumb: **what the app needs → code; what Claude needs to help you → MCP. Practical part — criteria:**

- ☐ Claude went through at least 2 scenarios via the browser MCP (entry, viewing)
- ☐ Confirmed the database state through the SQLite MCP after the scenarios
- ☐ The report contains what works / what doesn't / what's odd
- ☐ You personally verified at least one finding from the report (trust gets verified!)

Module 5 — solutions

Solution E5.1

Reference prompt:

Send an Explore subagent to investigate how this project handles errors (exceptions/error handling): where the central logic is, which error types it defines, how a user of the library receives errors. Have it return a short report with the paths of the key files.

Expected outcome: a 10–20 line report with concrete paths; your session didn't get "heavy" (the agent read in its own context). **Common mistake:** too fuzzy a task for the agent ("investigate the project") — an agent is at its BEST with a narrow, concrete question. A subagent doesn't see your conversation — everything it needs must be in the task.

Solution E5.2

Reference prompt:

Run 3 Explore subagents IN PARALLEL: (1) how tests are written and run in this project, (2) how configuration is organized, (3) which external dependencies it uses and what each is for. When they finish, merge the findings into one project overview for a new team member.

Expected outcome: three agents work simultaneously; you get a merged overview. Notice how much context it would have eaten to read all of that in your session.

Solution E5.3

Expected observation: direct reading = faster for SMALL questions (no agent startup "overhead"), but your context visibly fills up; a subagent = worth it when the answer requires reading many files, and when you need only the CONCLUSION and not the raw content. **Your rule of thumb (compare):** a question about 1–2 known files → direct; "how is X organized across the whole project" → a subagent; several independent questions → parallel subagents.

Solution E5.4

A complete example `.claude/agents/reviewer.md`:

```

---
name: reviewer
description: Read-only code review against a checklist. Use when a code
  review is requested. Returns a list of findings, changes nothing.
tools: Read, Grep, Glob
---

You are a strict but fair code reviewer. You NEVER change files –
you only read and report.

Review the code against this checklist:
1. Readability: do variable/function names say what they do?
2. Duplication: the same code in several places that should be extracted?
3. Obvious bugs: comparisons, loop bounds, null/undefined, edge cases.
4. Error handling: what happens when something goes wrong?
5. Dead code: functions/variables never used.

Report format – for each finding:
- [path:line] Short problem description → a concrete suggested fix
- Severity: BUG / RISK / STYLE

At the end: 2 sentences of overall impression. Don't report style
nitpicks that don't hinder understanding. At most the 10 most
important findings.

```

Why like this: `tools: Read, Grep, Glob` PHYSICALLY prevents it from changing anything (read-only is a constraint, not a request); the `description` says WHEN to use it (important for automatic invocation); the report format is fixed so output is comparable run to run; the findings limit prevents burying you.

Solution E5.5

Procedure: in the quiz folder invoke the reviewer ("run the reviewer agent on the whole project"). Expect 5–10 findings; typical for a vibe-coded quiz: duplicated DOM code, magic numbers (15 seconds written in 3 places), handling of an empty player name. **Criterion:** for every finding you made YOUR OWN decision (fix/ignore + why). The reviewer (and AI in general) proposes — you decide.

Solution E5.6

Success criteria:

- ☐ The three questions are YOURS and narrow enough for an agent to answer concretely
- ☐ Every finding in the report has file paths (no paths = retelling, not analysis)
- ☐ Your "hardest to change" assessment cites concrete findings (e.g. "logic X is spread across N files so every change touches all of them")

If the report has no paths: refine the agent's task — explicitly ask for paths. An agent does exactly what the task says.

Module 6 — solutions

Solution E6.1

Reference prompt:

Initialize a git repo. Create a .gitignore appropriate for [project type] and a CLAUDE.md with the project's basic rules ([your rules]). Then implement [first feature] and propose a commit – but show me the message before you commit.

What you check in the commit message: that it DESCRIBES WHAT the change does (not "changes" or "update"), short and concrete. Bad: "added files". Good: "Add task entry and listing with JSON storage".

Solution E6.2

Reference flow: Create a branch feature-X and implement: [description] → when done → /code-review → you read the findings → Fix findings 1 and 3, finding 2 is intentional → commit. **The point:** a review BEFORE the commit means a clean version remains in history. **Common mistake:** blindly fixing ALL findings — the review tool doesn't know your intentions; you judge which findings are justified.

Solution E6.3

Reference prompt: Publish this repo on my GitHub account as a public repo [name], push main and the branch, and open a Pull Request from the branch to main with a description of what the feature does and how it was tested. **Expected outcome:** the repo on GitHub; a PR with a meaningful description; after reviewing in the browser you merged it (you can also do it through Claude: "merge the PR"). **A prerequisite that's often forgotten:** gh auth login must be done before this.

Solution E6.4

Example .claude/settings.json **with a hook (Python project — a syntax check after every file edit):**

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "python -m py_compile \"$CLAUDE_FILE_PATHS\" && echo syntax OK"
          }
        ]
      }
    ]
  }
}
```

(Verify the exact event names and variables in the docs — `/help`, or ask Claude "set up a hook for me that checks the syntax of every edited .py file" and let it write `settings.json` itself — then *READ* what it wrote.) **Test:** request a code change → you see the hook fire in the session. Then deliberately ask it to introduce a syntax error — the hook should catch it. **Answer to the exercise question:** hooks are for rules that **MUST** hold (formatting, tests, no commits to main...) — CLAUDE.md is an instruction that gets read, a hook is a gate that can't be walked around.

Solution E6.5

A complete example `.claude/commands/ship.md`:

```
---
description: Check, commit and push the current changes
---
```

Do the following, in order, and stop if any step fails:

1. Run the project's checks if they exist (tests/linter);
if none exist, skip with a note.
2. Look at all current changes (git status and git diff).
3. Compose a short commit message that describes WHAT the changes do.
4. Show me the message and the file list, then wait for my confirmation.
5. After confirmation: commit and push to the current branch.

Argument (optional): \$ARGUMENTS – if provided, use it as the basis
for the commit message.

Usage: `/ship` or `/ship timer fix`. **Why step 4 exists:** a command that commits without showing you = automation without control; always leave a checkpoint at irreversible steps.

Solution E6.6

Example script (`daily-review.sh` / analogous for PowerShell):

```
#!/bin/sh
claude -p "Review the project: list all TODO and FIXME comments with
paths, group them by topic and propose the order to resolve them and
why. Change nothing." > daily-review.txt
```

Expected outcome: run by hand, it produces a usable report in a file. **The idea further:** a script like this in CI or a scheduler = Claude as part of the machinery, not just a conversation partner. Note: in headless mode think carefully about permissions — start with tasks that only READ.

Solution E6.7

Success criteria for the final exam — beyond the checklist in the task, qualitatively:

- ☐ You can show the repo to someone and explain every decision (why plan mode exactly there, why that MCP, what the hook guarantees)
- ☐ CLAUDE.md is such that a new person (or a new session) would continue the work without your verbal explanations — TEST IT: a new session + a request for a small improvement
- ☐ The commit history reads like the story of how the project came to be
- ☐ There is at least one moment where you REFUSED something Claude proposed, with a reason — that's the sign that you are leading, not the tool

If it all passed: congratulations — you've gone from "typing prompts" to "leading an agent through the whole software life cycle". That was the goal of the course.

This course is free to share. It was written against the publicly available Claude Code documentation (<https://code.claude.com/docs>); commands and names may change over time — when something doesn't work the way the course says, ask the tool itself (`/help`) or check the documentation.